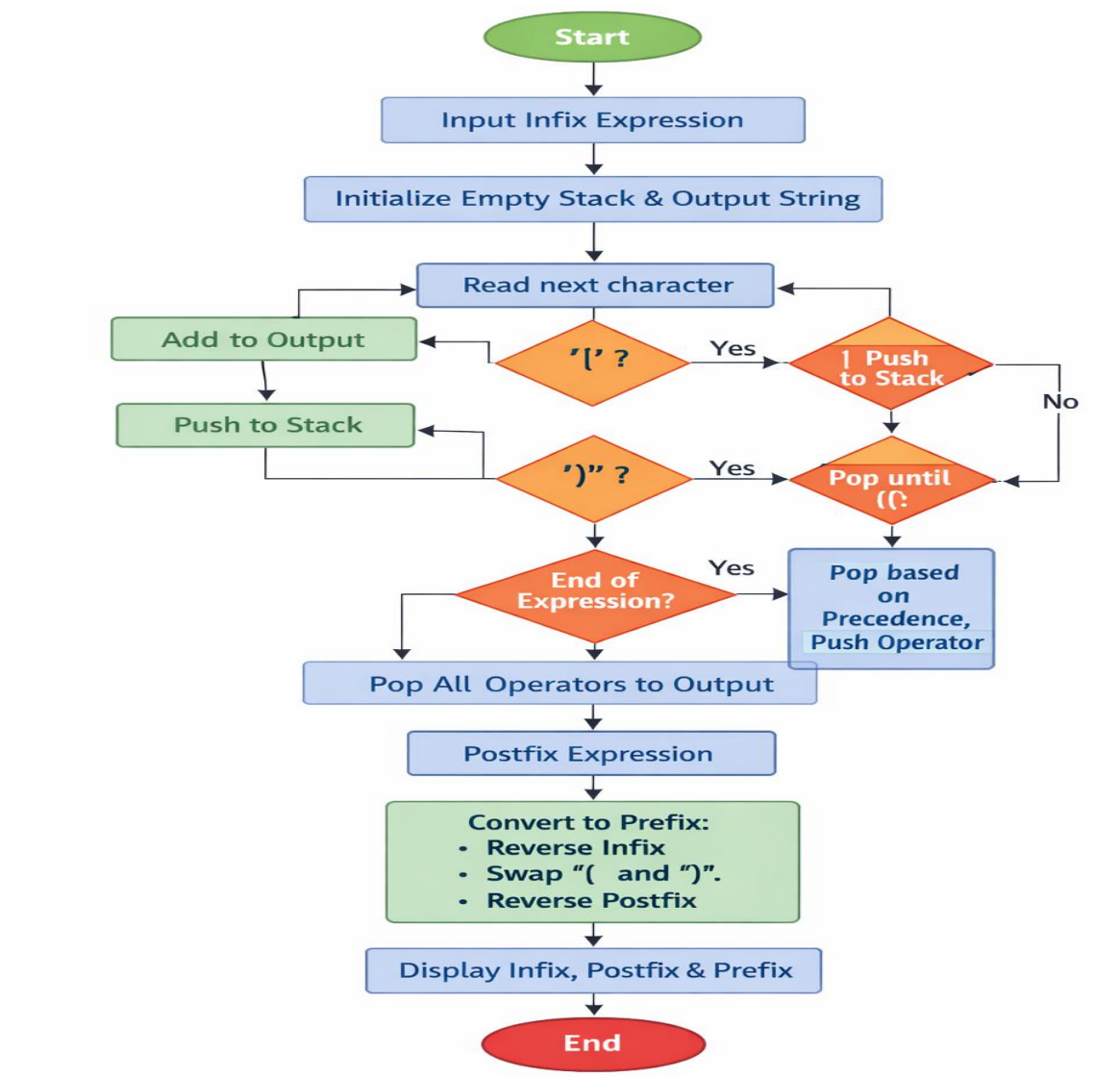
Assignment Title:

Conversion of Infix Expression to Postfix and Prefix Using Java

Abstract

This assignment demonstrates how to convert an infix expression into postfix and prefix forms using Java. It includes a detailed workflow represented by a flowchart, pseudocode outlining the algorithm, a complete Java program implementation, and example outputs showing the conversion results.

1. WorkFlow



2. Pseudocode

function infixToPostfix(expression):

create empty stack

create empty output string

```
    for each character in expression:
        if operand → add to output
        if '(' → push to stack
        if ')' → pop until '('
        if operator → pop higher/equal precedence operators, then push current
    pop remaining operators to output
    return output

function infixToPrefix(expression):
    reverse expression
    swap '(' and ')'
    postfix = infixToPostfix(reversed expression)
    prefix = reverse(postfix)
    return prefix
```

3. Java Program

```java
import java.util.Stack;


public class InfixConversion {


    static int precedence(char ch) {
        switch (ch) {
            case '+':
```

```
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
    }
    return -1;
}

static String infixToPostfix(String exp) {
    StringBuilder result = new StringBuilder();
    Stack<Character> stack = new Stack<>();

    for (int i = 0; i < exp.length(); i++) {
        char c = exp.charAt(i);

        if (Character.isLetterOrDigit(c)) {
            result.append(c);
        } else if (c == '(') {
            stack.push(c);
        } else if (c == ')') {
```

```
                while (!stack.isEmpty() && stack.peek() != '(') {
                    result.append(stack.pop());
                }
                stack.pop();
            } else {
                while (!stack.isEmpty() && precedence(stack.peek()) >=
precedence(c)) {
                    result.append(stack.pop());
                }
                stack.push(c);
            }
        }

        while (!stack.isEmpty()) {
            result.append(stack.pop());
        }

        return result.toString();
    }

    static String infixToPrefix(String exp) {
        StringBuilder reversed = new StringBuilder(exp).reverse();
        char[] chars = reversed.toString().toCharArray();
```

```
        for (int i = 0; i < chars.length; i++) {
            if (chars[i] == '(') chars[i] = ')';
            else if (chars[i] == ')') chars[i] = '(';
        }

        String postfix = infixToPostfix(new String(chars));
        return new StringBuilder(postfix).reverse().toString();
    }

    public static void main(String[] args) {
        String exp = "(A-B/C)*(A/K-L)";
        System.out.println("Infix: " + exp);
        System.out.println("Postfix: " + infixToPostfix(exp));
        System.out.println("Prefix: " + infixToPrefix(exp));
    }
}
```

4.Example Conversion

Expression: (A-B/C)*(A/K-L)

- (→ push
- A → output → A
- - → push
- B → output → AB
- / → push (higher precedence than -)

- C → output → ABC
-) → pop until ‘(‘→ output becomes ABC/-
- * → push
- (→ push
- A → output → ABC/-A
- / → push
- K → output → ABC/-AK
- - → push
- L → output → ABC/-AKL
-) → pop until (→ output becomes ABC/-AK/L-
- End: pop remaining * → final postfix = ABC/-AK/L-*
- Reverse logic → prefix = *-A/BC-/AKL

5. Example Output (from VS Code terminal)

Infix: (A-B/C)*(A/K-L)

Postfix: ABC/-AK/L-*

Prefix: *-A/BC-/AKL